



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



TFG del Grado en Ingeniería
Informática

TFG Topics

Análisis de temas en Trabajos Fin de
Grado



Presentado por Zeldan Javier Campos Cordero
en Universidad de Burgos — 4 de junio
de 2026

Tutor: Carlos López Nozal



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. Carlos López Nozal, profesor del departamento de Ingeniería Informática, área de Lenguajes y Sistemas Informáticos.

Expone:

Que el alumno D. Zeldan Javier Campos Cordero, con DNI 55462889E, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado Análisis de temas en Trabajos Fin de Grado.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 4 de junio de 2026

Vº. Bº. del Tutor:

D. Carlos López Nozal

Resumen

TFG Topics es una aplicación orientada al análisis temático cualitativo de la información textual asociada a los **Trabajo de Fin de Grado (TFG)**. El sistema se orienta a dos perfiles analíticos diferenciados. En el ámbito de la gestión académica, proporciona a docentes y estudiantes herramientas para mejorar la exploración de contenidos de **TFG**, permitiendo identificar las líneas temáticas asociadas a tutores específicos o a proyectos con mayores calificaciones. Por otro lado, en el ámbito de la investigación, facilita herramientas avanzadas para la construcción de modelos de temas orientadas a evaluar y comparar la coherencia metodológica de múltiples modelos.

Para alcanzar estos objetivos, se ha creado un conjunto de datos con 351 instancias correspondientes a **TFG** del Grado de Ingeniería Informática presentados en la Universidad de Burgos. Por cada proyecto se distinguen dos tipos de información: una académica, resultante de la gestión interna de la asignatura, y una técnica extraída de los repositorios de GitHub de los estudiantes, la cual incluye memorias en formato LaTeX, archivos *README*, *issues* y resúmenes.

Sobre este corpus se aplican técnicas de procesamiento del lenguaje natural (**NLP**) y la optimización mediante hiperparametrización en la construcción del modelado de temas. Se evalúan cuatro estrategias: una aproximación tradicional basada en bolsa de palabras mediante **Latent Dirichlet Allocation (LDA)** y tres enfoques basados en embeddings latentes: **Top2Vec**, **BERTopic** y **FastTopic**. Como resultado, se ha diseñado e implementado una aplicación web mediante el framework *Streamlit*. La plataforma permite analizar dinámicamente los temas de los distintos modelos obtenidos con la información del histórico de los **TFG** de la institución junto con la coherencia de los mismos. Un despliegue del frontend de la aplicación está disponible en <https://huggingface.co/spaces/zcc1001/tfg-topics>.

Descriptores

Análisis temático, Topic Modeling, LDA, BERTopic, Top2Vec, FASTopic, TFG,, Procesamiento del Lenguaje Natural, Streamlit.

Abstract

TFG Topics is an application for qualitative thematic analysis of textual information associated with Final Degree Project reports (TFG). The system targets two distinct analytical user profiles. In the context of academic management, it provides teachers and students with tools to improve exploration of TFG content, enabling identification of thematic lines linked to specific supervisors or to projects with higher grades. In the research context, it offers advanced tools for building topic models aimed at evaluating and comparing the methodological coherence of multiple models.

To achieve these objectives, a dataset of 351 instances was created, corresponding to TFG projects from the Computer Engineering degree at the University of Burgos. For each project, two types of information are distinguished: academic information, resulting from the course’s internal management, and technical information extracted from students’ GitHub repositories, including LaTeX reports, *README* files, *issues*, and abstracts.

On this corpus, natural language processing techniques (NLP) and hyperparameter optimization are applied in topic model construction. Four strategies are evaluated: a traditional bag-of-words approach based on Latent Dirichlet Allocation (LDA) and three latent embedding-based approaches: Top2Vec, BERTopic, and FastTopic. As a result, a web application was designed using the Streamlit framework. The platform enables dynamic analysis of topics from the different models obtained from the institution’s historical TFG data, together with their coherence. A deployment of the application frontend is available at <https://huggingface.co/spaces/zcc1001/tfg-topics>.

Keywords

Topic Modeling, LDA, BERTopic, Top2Vec, FASTopic, TFG, NLP, Streamlit.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
1.1. Materiales adjuntos	3
2. Objetivos del proyecto	5
2.1. Objetivo general	5
2.2. Objetivos específicos	5
2.3. Objetivos personales	6
3. Conceptos teóricos	9
3.1. Procesamiento de lenguaje natural	9
3.2. Preprocesamiento del textos	10
3.3. Representación del texto	14
3.4. Modelado de temas	15
3.5. Evaluación de modelos	19
3.6. Hiperparametrización de Modelos	21
3.7. Comparación de modelos	23
4. Técnicas y herramientas	25
4.1. Marco de trabajo	26
4.2. Plataforma de Gestión de Ciclo de Vida del Software (DevOps)	26
4.3. Herramientas de desarrollo	28

4.4. Documentación	30
5. Aspectos relevantes del desarrollo del proyecto	33
5.1. Arquitectura y diseño del software	33
5.2. Gestión de los datos y almacenamiento de resultados	34
5.3. Optimización de hiperparámetros y gestión de experimentos	35
5.4. Despliegue y modos de ejecución	35
5.5. Interfaz web y exploración de resultados	37
6. Trabajos relacionados	39
7. Conclusiones y Líneas de trabajo futuras	43
7.1. Líneas de trabajo futuras	44
Acrónimos	47
Glosario	49
Bibliografía	53

Índice de figuras

3.1. Pipeline general del preprocesamiento de lenguaje natural. . . .	11
4.1. Esquema de las principales tecnologías y librerías utilizadas. . .	25
5.1. Diagrama conceptual de los modos de ejecución del proyecto. . .	36
5.2. Vista general de la interfaz interactiva desarrollada con Streamlit.	38

Índice de tablas

1.1. Fuentes textuales incluidas en el alcance del proyecto	2
3.1. Comparativa de las características de los algoritmos de modelado de temas.	24
4.1. Flujo de desarrollo, control de versiones y validación continua. .	31
6.1. Resumen de trabajos relacionados	41

1. Introducción

En el ámbito universitario se genera cada curso una gran cantidad de documentos académicos que proporcionan una valiosa fuente de información. Entre ellos, las memorias de los **Trabajo de Fin de Grado (TFG)** que ocupan un lugar relevante, ya que forman parte fundamental del proceso formativo del estudiante, y permiten aplicar los conocimientos adquiridos durante la realización de los estudios de grado [39]. Al recopilar las tecnologías empleadas, los enfoques metodológicos adoptados y las líneas de desarrollo abordadas, estos documentos conforman una fiel radiografía del conocimiento generado en las diversas ramas de la ingeniería informática. Sin embargo, el principal problema es que esta información suele estar dispersa en repositorios y formatos heterogéneos, lo que dificulta su consulta sistemática.

Este trabajo tiene como finalidad el desarrollo de una aplicación orientada al análisis cualitativo de los textos asociados a **TFG** del Grado en Ingeniería Informática de la Universidad de Burgos. Permitiendo explorar, comparar resultados, detectar tendencias temáticas y facilitar la consulta de contenido académico a lo largo del tiempo. Para la consecución de los objetivos del sistema, se ha construido un conjunto de datos a partir del registro histórico de los **Trabajo de Fin de Grado (TFG)** defendidos en la Universidad de Burgos. Por cada proyecto integrado en el corpus, se distinguen y extraen dos tipos de información claramente diferenciados:

Metadatos de Gestión Académica: Información estructurada provista por los responsables de la asignatura, que contempla variables institucionales como el curso lectivo, la calificación final obtenida y la asignación de los tutores correspondientes.

Artefactos del Repositorio de Software: Información técnica y textual desarrollada por el estudiante en su repositorio remoto de GitHub.

Fuente textual	Uso principal en el proyecto
<i>readmes</i>	Descripción general del repositorio, tecnologías empleadas y contexto funcional del trabajo.
<i>issues</i>	Información complementaria sobre incidencias, decisiones técnicas y evolución del desarrollo.
<i>memorias TFG</i>	Contenido textual completo de las memorias en formato $\text{\LaTeX}$.
<i>abstracts</i>	Resúmenes académicos para análisis comparativo de temas de forma directa.

Tabla 1.1: Fuentes textuales incluidas en el alcance del proyecto

Este flujo de datos abarca el código fuente de las memorias en formato $\text{\LaTeX}$, los archivos *README*, el histórico de incidencias (*issues*) y los resúmenes (*abstracts*) del proyecto. Se muestra una descripción más detallada de los estos elementos documentales asociados a los **TFG** en la Tabla 1.1.

Sobre este corpus se aplican cuatro técnicas de modelado de temas, una clásica basada en **Bag of Words (BOW)** (**Latent Dirichlet Allocation (LDA)** [28]), y tres enfoques basados en *embeddings* (**Top2Vec** [32], **BERTopic** [36] y **FASTopic** [45]). El propósito no es únicamente obtener resultados automáticos, sino también comparar el comportamiento de los modelos y estudiar su utilidad en un contexto académico real. La comparación de modelos implica procesos de construcción basados en la optimización de parámetros y medidas de coherencia.

La solución implementada se basa en una arquitectura hexagonal con tres módulos: ingesta de datos, procesamiento y visualización. Esta separación permite desacoplar responsabilidades, mejorar la mantenibilidad del sistema y facilitar su reproducibilidad en distintos entornos de ejecución.

Desde el punto de vista teórico, este proyecto se fundamenta en la aplicación de técnicas de **Modelado de temas** (*Topic Modeling*), una subárea del aprendizaje no supervisado cuyo propósito es identificar la estructura temática latente en un conjunto de documentos [25]. Estos modelos matemáticos parten del supuesto de que cada documento puede representarse como una combinación finita de temas, donde cada tema se define, a su vez, como una distribución de probabilidad sobre un vocabulario de términos.

La utilidad analítica de esta abstracción probabilística se proyecta directamente sobre los distintos perfiles de usuario del sistema:

Tutores y Estudiantes: Se benefician de la capacidad del modelo para condensar y organizar grandes corpus textuales no estructurados. Esto les permite optimizar la exploración y búsqueda semántica de información académica, facilitando la identificación de las líneas de trabajo asociadas a directores específicos o la categorización de los proyectos con mayor rendimiento académico.

Investigadores: Encuentran su utilidad en la granularidad del análisis de temas. Al examinar la distribución e importancia probabilística de los términos, los investigadores disponen de las herramientas necesarias para evaluar y comparar la coherencia entre múltiples estrategias de construcción de modelos de temas, permitiendo el descubrimiento y seguimiento de tendencias científicas emergentes.

Esta memoria se estructura del siguiente modo: en **Objetivos del proyecto** se definen el objetivo general y los objetivos específicos; en **Conceptos teóricos** se presenta el marco conceptual del modelado de temas; en **Técnicas y herramientas** se describen las tecnologías utilizadas; en **Aspectos relevantes del desarrollo del proyecto** se detallan la metodología seguida, las decisiones técnicas y las dificultades abordadas; en **Trabajos relacionados** se analizan propuestas similares y se compara su enfoque con el de este trabajo; y, por último, en **Conclusiones y líneas de trabajo futuras**, se recogen los resultados obtenidos y las posibles extensiones del sistema.

1.1. Materiales adjuntos

A continuación una lista con los recursos entregados junto con esta memoria:

- Repositorio del proyecto: <https://github.com/zcc1001/tfg-topic>
- Wiki del proyecto: <https://github.com/zcc1001/tfg-topics/wiki>
- Despliegue de la aplicación: <https://huggingface.co/spaces/zcc1001/tfg-topics>

2. Objetivos del proyecto

El presente Trabajo Fin de Grado tiene como objetivo principal el desarrollo de una aplicación capaz de realizar análisis temáticos cualitativos sobre los textos contenidos en las memorias de los Trabajo de Fin de Grado (TFG) en Ingeniería Informática. Las memorias, archivos *README*, el histórico de incidencias (*issues*), los resúmenes (*abstracts*) del proyecto y demás documentos descriptivos del TFG se encuentran disponibles en repositorios de software alojados en la plataforma GitHub.

Este proyecto busca aplicar técnicas de procesamiento del lenguaje natural y modelado de temas para facilitar el análisis, explorar y comparar resultados, detectar tendencias temáticas y facilitar la consulta de contenido académico a lo largo del tiempo.

2.1. Objetivo general

Diseñar e implementar una aplicación que permita realizar análisis temáticos automáticos sobre la documentación disponible en los repositorios del TFG, como archivos *README*, *issues*, *abstracts* y memorias en formato $\text{\LaTeX}$ utilizando para ello distintos algoritmos de modelado de temas y evaluando su rendimiento en términos de coherencia y calidad temática.

2.2. Objetivos específicos

Para alcanzar el objetivo general, se plantean los siguientes objetivos específicos:

- **Recopilar y preparar los datos:** Construir un conjunto de datos a partir de **TFG** disponibles públicamente en GitHub, integrando las fuentes *issues*, *readmes*, memorias y *abstracts*.
- **Implementar diferentes algoritmos de modelado de temas:** Aplicar y comparar una aproximación clásica basada en bolsa de palabras (**LDA**) frente a tres enfoques basados en *embeddings* (**Top2Vec**, **BERTopic** y **FASTopic**).
- **Evaluar el rendimiento de los modelos:** Medir y comparar la calidad de los temas generados mediante métricas de coherencia, junto con el tiempo de construcción de cada modelo para analizar su coste computacional.
- **Desarrollar una interfaz visual:** Diseñar una aplicación que permita la exploración interactiva de resultados por modelo, conjunto de datos y métricas.
- **Fomentar la gestión del conocimiento académico:** Estructurar y facilitar el acceso al conocimiento generado en **TFG** de cursos anteriores integrando variables académicas (curso, calificación, tutores y fecha de presentación) con contenido técnico del proyecto, con el fin de apoyar la toma de decisiones en docencia e investigación y favorecer la reutilización del conocimiento institucional.
- **Diseño de arquitectura:** Diseñar e implementar una arquitectura de software modular que separe ingesta, procesamiento y visualización, siguiendo principios de Ingeniería del Software.
- **Gestión del desarrollo avanzado del software:** Aplicar herramientas y prácticas ágiles para planificar y hacer seguimiento del proyecto, utilizando GitHub y Zube, junto con prácticas de integración continua, testing continuo, *release* continuo y despliegue continuo mediante Docker y Hugging Face.

2.3. Objetivos personales

- Crear una herramienta que sea de utilidad para la comunidad universitaria.
- Aplicar conocimientos adquiridos durante el grado, especialmente en el ámbito de Desarrollo Avanzado de Sistemas Software y Minería de Datos.

- Consolidar conocimientos en las siguientes tecnologías: Python, Git y Docker.
- Mejorar habilidades de diseño de software, aplicación de buenas prácticas en desarrollo, integración y despliegue continuo, testing de software y documentación.

3. Conceptos teóricos

A continuación se exponen conceptos teóricos que sientan las bases del proyecto, cubriendo desde el procesamiento de lenguaje natural hasta los algoritmos avanzados de modelado de temas.

3.1. Procesamiento de lenguaje natural

El procesamiento de lenguaje natural (**NLP**, del inglés **Natural Language Processing**) es un campo de la Inteligencia Artificial (**AI**, del inglés **Artificial Intelligence**) que se ocupa del análisis y la interpretación de textos que se encuentran en lenguaje humano. Utilizando estas técnicas los computadores pueden dar sentido a los datos contenidos en un conjunto de textos y así adquirir su conocimiento implícito [24].

NLP usa gran variedad de técnicas y modelos para analizar diferentes aspectos de los textos como la formación de palabras, el léxico, la sintaxis entre otros, con esto se pretende resolver problemas específicos relacionados con el lenguaje.

Entre las técnicas más usadas en **NLP** se encuentran la **Tokenización**, **Lematización**, eliminación de palabras vacías (del inglés **Stopwords**), análisis sintáctico y extracción de información. En el contexto de este trabajo, las técnicas de **NLP** se utilizan para preparar los textos de las memorias de **TFG**, transformándolos en datos estructurados que puedan ser analizados mediante algoritmos de modelado de temas.

Conjunto de datos para el modelado temático

Los modelos de temas de este proyecto se construyen a partir de un conjunto de datos del registro histórico de los **Trabajo de Fin de Grado (TFG)** defendidos en la Universidad de Burgos. Dicho conjunto de datos se organiza en dos bloques de información claramente diferenciados: **Metadatos de Gestión Académica** y **Artefactos del Repositorio de Software**. Además, el corpus de trabajo está formado por 351 instancias de **TFG**.

Los **Metadatos de Gestión Académica** recogen la información estructurada del contexto docente (curso lectivo, fechas de asignación y presentación, calificación final, tutores y estudiantes). Este bloque permite segmentar y comparar resultados por criterios académicos, por ejemplo para estudiar la evolución temporal de temáticas, analizar la distribución de líneas de trabajo por tutor o relacionar temas con rangos de calificación.

Por su parte, los **Artefactos del Repositorio de Software** constituyen la evidencia textual y técnica del trabajo desarrollado en cada **TFG** (memoria en **L^AT_EX**, *README*, *issues* y *abstracts*). Este bloque es la base lingüística sobre la que se aplican las técnicas de **NLP** y los algoritmos de modelado de temas para extraer patrones semánticos, términos representativos y estructuras temáticas latentes.

Para una descripción completa del dominio, términos y requisitos asociados al conjunto de datos (actores, fuentes, restricciones y casos de uso), véase el apéndice de Especificación de Requisitos.

3.2. Preprocesamiento del textos

Antes de iniciar cualquier análisis los textos deben someterse a una serie de procesos secuenciales para garantizar su limpieza, normalización y reducir la dimensión del vocabulario, con el fin de garantizar la calidad de los análisis resultantes. El flujo general de estas transformaciones se ilustra en la Figura 3.1.

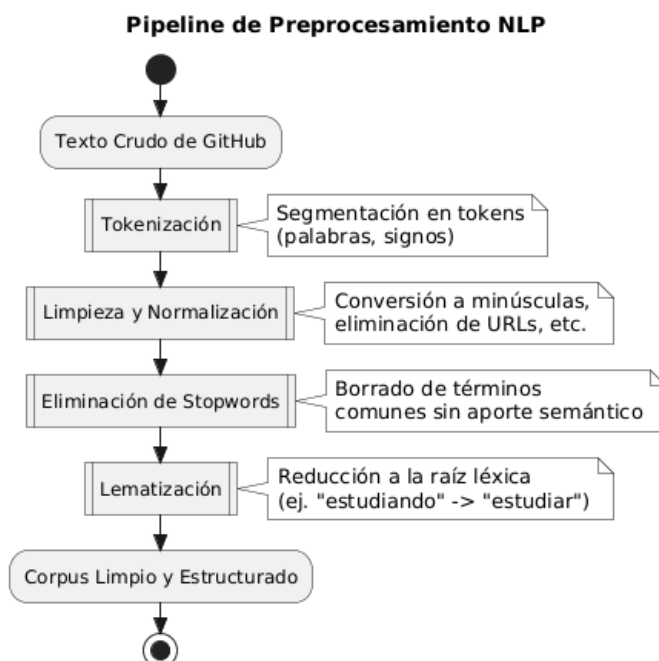


Figura 3.1: Pipeline general del preprocesamiento de lenguaje natural.

Tokenización

La **Tokenización** es un proceso fundamental dentro de el **NLP** y la **AI**, pues permite dividir el texto en unidades mínimas denominadas tokens, que pueden ser palabras, subpalabras, caracteres o unidades semánticas mas complejas [38]. Este proceso constituye la base para la mayoría de técnicas posteriores de análisis textual, ya que permite transformar cadenas de texto sin estructura en secuencias manejables por algoritmos computacionales.

La **Tokenización** presenta una serie de problemas que son específicos de cada idioma. Por lo tanto, se requiere conocer el idioma del documento. La identificación del idioma mediante clasificadores que usan subsecuencias cortas de caracteres como características resulta muy efectiva, ya que la mayoría de los idiomas tienen patrones distintivos [11].

Tipos de Tokenización

Existen varios métodos comunes para realizar esta segmentación

Tokenización de palabras El texto se divide en palabras individuales basándose en un delimitador, típicamente el espacio. Esta representación puede fallar con palabras compuestas o en idiomas sin espacios entre palabras (como el chino) [34].

Tokenización de caracteres El texto se divide en caracteres individuales. Este método es útil para lenguajes con alfabetos complejos y permite al modelo manejar un rango más amplio de entradas, como palabras desconocidas o errores tipográficos [29, 34].

Tokenización de subpalabras El texto se divide en subpalabras o fragmentos frecuentes de caracteres. Este método reduce la dimensionalidad y maneja eficazmente palabras raras y desconocidas [29, 34].

Tokenización de espaciado blanco Divide el texto basándose únicamente en los espacios, lo que resulta en una gestión incorrecta de puntuaciones o errores gramaticales [29].

En el contexto del análisis temático de TFG, la **Tokenización** cumple varias funciones clave:

- Estandarizar los textos, permitiendo compararlos entre sí.
- Reduce ruido, al separar puntuación, símbolos o caracteres no relevantes.
- Facilita operaciones posteriores, como **Lematización**, extracción de n-grams o cálculo de frecuencias.
- Mejora la precisión del modelado de temas, ya que una buena segmentación permite que los modelos identifiquen patrones semánticos coherentes.

En particular para este trabajo la **Tokenización** presenta varios problemas o desafíos:

Tratamiento de compuestos palabras como “aprendizaje-servicio” deben decidirse como uno o dos tokens.

Abreviaturas universitarias “TFG”, “EPS”, “ECTS”, etc., que pueden requerir reglas específicas.

Símbolos técnicos o matemáticos comunes en textos de informática.

Preservación del contexto eliminar puntuación puede afectar el significado y pérdida de contexto en algunos casos.

La **Tokenización** es un proceso esencial y su correcta implementación afecta directamente la calidad del análisis temático y de los modelos posteriores, desde la extracción de palabras clave hasta el modelado de temas.

Lematización

La **Lematización** es una técnica clave dentro del **NLP** que tiene como principal objetivo transformar o reducir las palabras a su forma canónica o lema, es decir, la forma bajo la cual se encontrarían en un diccionario. En muchas lenguas, el infinitivo se utiliza como lema para el verbo; por ejemplo, en español, dormir es el lema para la forma duermes [38].

El lema (*lemma*) se define como la forma de citación o forma base de una palabra. El concepto del lema es relevante para comprender la estructura del significado. Un lema puede ser polisémico, es decir, puede tener múltiples sentidos. Cada aspecto individual del significado de un lema se denomina sentido de palabra (*word sense*). Por ejemplo, el lema ratón tiene al menos dos sentidos distintos: El roedor o el dispositivo de control de cursor [38].

La **Lematización**, al reducir las palabras a su forma base, asegura que un sistema de **NLP** trate variantes flexionales (como singular, plural o diferentes tiempos verbales) como la misma unidad, facilitando la generalización sobre el significado fundamental de la palabra.

Stopwords

Las palabras vacías (del inglés **Stopwords**) es un termino que hace referencia a las palabras que deben ser eliminadas en la fase de **Preprocesamiento de texto** para garantizar la precisión y eficiencia de las tareas de **NLP**. Estas palabras aparecen frecuentemente en muchos documentos, pero transmiten poca o nula información sobre la sección del texto al que pertenecen [42].

Al eliminar estas palabras que aportan baja información, se incrementa la significación estadística de los términos que si son relevantes y son verdaderamente importantes para la tarea. Ejemplos específicos de estas palabras son: cada, sobre, tal, el, la, entre otros [42].

Históricamente, se han realizado esfuerzos para identificar **Stopwords** a partir de fuentes de conocimiento genéricas, como *Brown Corpus* o *20 newsgroup corpus*. Esto ha provocado la creación de listas genéricas que

se utilizan en aplicaciones de **NLP**. Una de las listas más utilizadas es la distribuida con el paquete de Python **NLTK** (del inglés, **Natural Language Tool Kit**) [42].

Sin embargo, el lenguaje técnico utilizado en textos de ingeniería o tecnología se distingue del lenguaje común. El lenguaje técnico contiene su propio argot y palabras altamente frecuentes y poco informativas que no están presentes en las listas genéricas. No existe una lista estándar de **Stopwords** para aplicaciones de procesamiento de lenguaje técnico. La adopción simple de listas genéricas deja muchos términos repetitivos y poco informativos específicos del dominio en los datos. Es por esto que el proceso de identificación de **Stopwords** incluye una etapa posterior de evaluación humana por parte de expertos para confirmar que el término no contiene información sobre ingeniería y tecnología [42].

3.3. Representación del texto

Bolsa de palabras

La bolsa de palabras (**BOW**, inglés **Bag of Words**) es una de las técnicas de representación de texto utilizada por distintas aplicaciones de **NLP**. Esta representación se basa en el conteo de palabras, ignorando el orden y la estructura gramatical del texto [37].

El método **BOW** implica la construcción de un vocabulario compuesto por todas las palabras únicas presentes en los documentos analizados y la definición de una métrica concreta para determinar el peso de cada término. Las métricas más empleadas para este fin son la frecuencia de términos (**TF**, inglés **Term Frequency**), y la frecuencia inversa de documentos (**TF-IDF**, del inglés **Term Frequency-Inverse Document Frequency**) [37].

Una de las principales limitaciones de **BOW** es la pérdida de información contextual de los términos dentro de los documentos. Otro inconveniente de **BOW** es que no captura el significado de las palabras y trata todas las palabras como entidades independientes dentro del documento. Además, la vectorización mediante **BOW** genera vectores dispersos de alta dimensión cuando se trabaja con **Corpus** extensos, lo que incrementa la complejidad computacional y los requisitos de memoria. **BOW** no es capaz de gestionar los sinónimos, tratando palabras con un mismo significado como si estuvieran relacionadas [37].

Representaciones vectoriales

Debido a las limitaciones encontrados en otras formas de representación de texto como **BOW**, han surgido técnicas que buscan solucionar estos problemas, entre las cuales podemos encontrar a los **Embeddings**. Los **Embeddings** son una técnica de procesamiento de lenguaje natural que convierte el lenguaje humano en vectores matemáticos. Estos vectores son una representación del significado subyacente de las palabras, lo que permite que las computadoras procesen el lenguaje de manera más efectiva [30].

Los **Embeddings** intentan solucionar el problema de la pérdida de contexto y semántica de **BOW** considerando el orden y la secuencia de aparición de cada término. Estos modelos se construyen sobre redes neuronales artificiales (**ANN**, inglés **Artificial Neural Networks**) y se entrenan con grandes cantidades de datos para capturar el significado contextual de una unidad léxica en función de las palabras que la rodean, y viceversa. Cada palabra en el espacio de **Embeddings** se representa como un vector de números reales con una dimensión predefinida. En un espacio de vectores de palabras, es posible identificar las palabras más similares a un término concreto del vocabulario utilizando métodos como la **Similitud del coseno** o la **Distancia euclidiana**. Estas técnicas permiten agrupar palabras relacionadas, clasificar textos, categorizar documentos y descubrir temas o temas latentes dentro del texto [37].

3.4. Modelado de temas

El **Modelado de temas** (del inglés *Topic Modeling*) es una técnica muy utilizada con la cual podemos encontrar temas subyacentes dentro de un grupo de documentos que deseemos analizar. Esta sigue un enfoque estadístico y proporciona una forma de representar textos de una manera estructurada, lo que la hace ideal para analizar grandes conjuntos de datos textuales [27].

Dependiendo del algoritmo utilizado los términos se pueden obtener utilizando distintos enfoques como la bolsa de palabras **BOW**, frecuencia de términos, frecuencia inversa de términos o basado en clases [40].

Podemos destacar la importancia del modelado de temas en los siguientes aspectos [27]:

Organización de la información permite categorizar los documentos en temas lo cual hace más fácil el proceso de búsqueda.

Resumen permite capturar la esencia de la temática del documento.

Recomendación facilita realizar recomendaciones personalizadas para cada usuario usuarios

En este **TFG** nos centraremos en el uso de los siguiente modelos:

Latent Dirichlet Allocation (LDA) Es un modelo probabilístico clásico que data de 2003. Se basa en el enfoque de bolsa de palabras (**BOW**) y asume que los documentos son una mezcla de temas y los temas una mezcla de palabras. Este es una referencia fundamental en el modelado de temas [28]. En la sección 3.4 se profundiza sobre este modelo.

Top2Vec Surge en 2020 como una alternativa que utiliza **Embeddings** de palabras y documentos para encontrar temas. Este crea un espacio semántico unificado donde los vectores de temas, documentos y palabras coexisten, permitiendo una búsqueda semántica y la detección automática del número de temas [22]. En la sección 3.4 se profundiza sobre este modelo.

BERTopic También de 2020, este modelo aprovecha el poder de los **Embeddings** contextuales de los transformadores (como **BERT**). Su arquitectura modular combina la **Reducción de dimensionalidad** con **UMAP** y el **Clustering** con **HDBSCAN** para identificar temas semánticamente coherentes, ofreciendo una gran flexibilidad [36]. En la sección 3.4 se profundiza sobre este modelo.

FASTopic Es el modelo más reciente, de 2023, y se presenta como una evolución de los modelos basados en **Embeddings**. Introduce un enfoque optimizado para ser más rápido, estable y escalable, manteniendo la alta calidad de los temas. Está diseñado para funcionar eficientemente en grandes volúmenes de datos [45]. En la sección 3.4 se profundiza sobre este modelo..

Latent Dirichlet Allocation (LDA)

Latent Dirichlet Allocation (LDA) es un modelo probabilístico que usa matrices de factorización basado en la distribución de Dirichlet. En este modelo se asume que cada documento expone una serie de temas y cada tema se caracteriza por una distribución de palabras basada en la frecuencia de aparición en el documento. Los resultados obtenidos al ejecutar el modelo no son deterministas, por lo que se puede obtener un resultado distinto en cada ejecución para el mismo conjunto de datos [40, 27].

LDA posee dos parámetros [26] que debemos ajustar para mejorar los resultados obtenidos:

- Numero de temas
- Factor de densidad del documento-tema
- Factor de densidad tema-palabra

Top2Vec

Top2Vec es un algoritmo de modelado de temas basado en **Embeddings**, documentos y vectores de palabras. Este algoritmo asume que la semántica interna de los documentos es indicativo de que existe una temática común [32].

A diferencia de otros métodos como LDA, Top2Vec genera simultáneamente **Embeddings** de documentos, **Embeddings** de palabras y **Embeddings** de temas, lo que le permite identificar agrupaciones semánticas de forma más natural y continua. Este enfoque se basa en la idea de que los documentos que tratan temas similares deben estar próximos entre sí en un espacio vectorial de alta dimensión. Mediante el uso de modelos de lenguaje basados en redes neuronales, Top2Vec representa documentos y palabras en un espacio semántico compartido, permitiendo descubrir temas como densidades locales o agrupaciones (del inglés *clusters*) de documentos [22].

Ventajas de Top2Vec [22]:

- No requiere preprocesamiento, ya que funciona bien sin necesidad de: eliminación de **Stopwords**, realizar **Lematización**, limpiar puntuación, filtrar términos raros, etc.
- No requiere elegir el número de temas.
- Produce temas con alta **Coherencia** semántica.
- Permite buscar documentos por similitud, identificar documentos mas representativos de un tema.

BERTopic

BERTopic es un método de modelado de temas que combina **Embeddings** semánticos, **Reducción de dimensionalidad** y **Clustering** para identificar temas en grandes colecciones de texto de manera automática y precisa [36].

BERTopic se basa en el uso de **Bidirectional Encoder Representations from Transformers (BERT)** para generar representaciones vectoriales **Embeddings** de los documentos y temas. El proceso se ejecuta fundamentalmente en dos fases: la codificación semántica y la extracción de temas. En la primera fase, los documentos se transforman en **Embeddings** de alta dimensionalidad mediante **BERT**. Posteriormente, se emplea el algoritmo **HDBSCAN** para agrupar estos vectores y así identificar clústeres en el espacio vectorial, permitiendo la clasificación de los textos en distintos temas. A partir de cada grupo, se derivan las palabras clave que describen el contenido [33].

BERTopic combina cuatro componentes principales:

- **Embeddings** (representaciones semánticas)
- **Reducción de dimensionalidad (UMAP)**, una técnica que reduce las dimensiones de los **Embeddings** manteniendo las relaciones importantes.
- **Clustering** basado en densidad (**HDBSCAN**), es un algoritmo que detecta “grupos naturales” en los datos.
- Representación final de temas (**c-TF-IDF**), método para identificar las palabras más representativas de cada tema.

La principal ventaja de BERTopic es su capacidad para encontrar matices semánticos y manejar grandes volúmenes de información textual. Puede identificar significados complejos y relaciones contextuales con mayor precisión. Además, a diferencia de otros métodos, no es necesario fijar de antemano cuántos temas se generarán, ya que el algoritmo de **Clustering** determina este número automáticamente [33].

FASTopic

FASTopic (*Fast, Adaptive, Stable and Transferable topic model*). Es un modelo que sigue el enfoque denominado **Reconstrucción de Relaciones Semánticas Dual (DSR)**, este no utiliza redes neuronales y se basa en tres elementos: la representación vectorial de los documentos generados por un transformador predeterminado, los vectores de temas y de palabras [45].

DSR captura dos tipos de relaciones semánticas: entre documentos y temas, y entre temas y palabras, interpretándolas como distribuciones que permiten descubrir los temas de manera ordenada y eficiente, eliminando la necesidad de estructuras complicadas. Para modelar estas relaciones,

se propone además el **Embedding Transport Plan (ETP)**, un mecanismo novedoso que, en lugar de emplear funciones *softmax* parametrizadas, regula estas relaciones como planes de transporte óptimo entre los **Embeddings** de documentos, temas y palabras. Esto reduce el sesgo en las relaciones y permite obtener temas bien diferenciados y distribuciones más precisas [45].

FASTopic se basa en los siguientes pasos:

- Generación de **Embeddings** (representaciones semánticas).
- **Reducción de dimensionalidad**.
- **Clustering**, no se limitan a **HDBSCAN**, sino que permiten el uso de múltiples algoritmos de **Clustering** que buscan eficiencia y escalabilidad.
- Extracción de palabras representativas del tema.

FASTopic tiene como principales ventajas la capacidad de manejar documentos de gran tamaño si necesidad de utilizar mayor capacidad de procesamiento, no requiere limpieza extrema del texto y produce una calidad semántica similar a la de BERTopic cuando se usan buenos **Embeddings**.

3.5. Evaluación de modelos

Evaluar la calidad del resultado de los modelos es una parte esencial del proceso de análisis. Es importante verificar si los temas generados son coherentes, útiles y representativos para los textos.

La evaluación puede realizarse utilizando alguna de las siguientes técnicas:

- Utilizando métricas automáticas, que proporcionan valores numéricos objetivos.
- Evaluación humana o cualitativa, que revisa si los temas tienen sentido.
- Utilizando métricas híbridas, que combinan ambas aproximaciones.

Métricas automáticas

Las métricas permiten comparar el resultado de los modelos sin necesidad de intervención humana, las más comunes son: perplejidad y coherencia.

Perplejidad

Es una medida utilizada en modelos probabilísticos como **LDA**. indica la capacidad del modelo para predecir documentos no vistos. Cuanto mas baja mejor se considera el desempeño del modelo, implica que el modelo tiene una mayor capacidad para anticipar las palabras de textos que no han sido utilizados durante el entrenamiento.

Solo es útil para comparar versiones diferentes del mismo tipo de modelo probabilístico (**LDA**). No es adecuada para modelos basados en **Embeddings** (BERTopic, Top2Vec, FASTopic).

Coherencia

Es una métrica que evalúa como de relacionados están los términos de las palabras más representativas de un tema. Esta métrica permite identificar cuantos temas son adecuados para un conjunto de datos determinado [41].

La **Coherencia** busca medir hasta qué punto las palabras más representativas de un tema tienden a aparecer juntas en el **Corpus** y a mantener relaciones semánticas coherentes. Una mayor **Coherencia** indica un tema más interpretable y semánticamente consistente.

Los métodos más usados son:

- **C_V**: basada en similitud semántica.
- **UCI**: mide coocurrencias de palabras en **Ventana deslizante**.
- **UMass**: calcula coocurrencias dentro del propio **Corpus**.

Esta métrica funciona bien tanto en **LDA** como en BERTopic, Top2Vec o FASTopic.

Evaluación Humana

El investigador revisa la lista de términos que componen cada tema y determina si son coherentes. En investigación cualitativa es habitual que varias personas evalúen los resultados y se calcule el índice de acuerdo inter-evaluador. Esto ayuda a evitar que la evaluación dependa de la subjetividad de una sola persona [44]

3.6. Hiperparametrización de Modelos

Los hiperparámetros son valores configurados por el investigador antes de entrenar un modelo. Determinan el comportamiento del algoritmo, la calidad de los temas obtenidos y el tiempo de cómputo. Ajustarlos adecuadamente es esencial para obtener resultados óptimos en el análisis temático [28, 35].

Hiperparámetros en LDA

LDA es uno de los modelos más sensibles a su configuración.

Número de temas (k)

Controla cuántos temas debe encontrar el modelo. Valores demasiado bajos generan temas demasiado generales, mientras que valores demasiado altos producen temas ruidosos o redundantes.

Parámetro α

Regula cuántos temas aparecen en cada documento. Valores altos implican documentos con muchos temas mezclados; valores bajos generan documentos dominados por pocos temas.

Parámetro β

Determina cuántas palabras componen cada tema. Un valor alto genera temas más dispersos y un valor bajo produce temas más específicos.

Número de Iteraciones

Afecta la estabilidad del modelo. Más iteraciones proporcionan mejor convergencia a costa de un mayor tiempo de entrenamiento.

Hiperparámetros en BERTopic

BERTopic tiene hiperparámetros en varias etapas: **Embeddings**, **Reducción de dimensionalidad** (UMAP), **Clustering** (HDBSCAN) y extracción de palabras mediante **c-TF-IDF**.

UMAP

Los hiperparámetros más relevantes son:

- **n_neighbors**: controla la influencia del vecindario local. Valores bajos encuentran temas más específicos.
- **n_components**: número de dimensiones finales.

HDBSCAN

Los ajustes más influyentes son:

- **min_cluster_size**: tamaño mínimo del grupo.
- **min_samples**: sensibilidad al ruido.

Hiperparámetros en Top2Vec

Top2Vec utiliza hiperparámetros principalmente en el modelo de **Embeddings** (**Doc2Vec**, **BERT** o **USE**) y en configuraciones de **UMAP** y **HDBSCAN** compartiendo muchos de los ajustes de **BERTopic**.

Hiperparámetros en FASTopic

FASTopic está optimizado para grandes volúmenes de datos y permite configurar:

- tipo de embeddings (**BERT**, **FastText**, etc.).
- número de clusters en métodos rápidos como **K-Means**.
- número de palabras clave extraídas por tema.

Tanto la evaluación como la **Hiperparametrización** de modelos de temas son procesos fundamentales para garantizar la calidad del análisis temático. Modelos como LDA requieren ajustes cuidadosos en parámetros probabilísticos, mientras que modelos modernos como **BERTopic**, **Top2Vec** y **FASTopic** dependen más de configuraciones de **Embeddings**, **Reducción de dimensionalidad** y **Clustering**. La combinación de métricas automáticas y evaluación humana proporciona la estrategia más robusta para obtener temas coherentes y útiles en contextos como el análisis de **TFG**.

Métodos de búsqueda de hiperparámetros

Existen diversas estrategias para ajustar hiperparámetros, con distintos niveles de complejidad y eficiencia:

- **Grid Search:** explora todas las combinaciones posibles dentro de un conjunto definido. Sin embargo, resulta poco eficiente cuando el número de hiperparámetros es elevado [31].
- **Random Search:** selecciona combinaciones aleatorias de hiperparámetros dentro de los rangos definidos. Ha demostrado ser más eficiente que la búsqueda exhaustiva en espacios de gran dimensionalidad [31].
- **Optimización Bayesiana:** construye un modelo probabilístico del rendimiento y selecciona nuevas combinaciones basadas en regiones prometedoras del espacio de búsqueda. Es una opción muy eficiente cuando las evaluaciones del modelo son costosas [43].

3.7. Comparación de modelos

Con el objetivo de visualizar las diferencias de los distintos modelos seleccionados, en la Tabla 3.1 se presentan las propiedades de los cuatro algoritmos de modelado de temas evaluados. Esta comparación permite visualizar la evolución desde los modelos probabilísticos clásicos hacia los más recientes.

Características	LDA	Top2Vec	BERTopic	FASTopic
Requiere indicar número de temas k	Sí	No	No	No
Uso de <i>embeddings</i>	No	Sí	Sí	Sí
Representación del texto	BOW	<i>embeddings</i> semánticos	<i>embeddings</i> contextuales	<i>embeddings</i> contextuales
Preprocesamiento necesario	Alto	Muy bajo	Moderado	Bajo
Coherencia de temas	Media	Muy Alta	Alta	Muy Alta
Coste computacional	Bajo	Alto	Alto	Alto
Interpretabilidad	Alta	Media	Media-Alta	Media

Tabla 3.1: Comparativa de las características de los algoritmos de modelado de temas.

4. Técnicas y herramientas

En este capítulo se describen las técnicas y herramientas utilizadas durante el desarrollo del proyecto. Se abordan tanto las metodologías empleadas como las herramientas de software que facilitaron la implementación y gestión del mismo.

En la Figura 4.1 se presenta un resumen conceptual del ecosistema tecnológico empleado, categorizado por su función principal dentro de la arquitectura.



Figura 4.1: Esquema de las principales tecnologías y librerías utilizadas.

4.1. Marco de trabajo

El desarrollo de *TFG Topics* se llevó a cabo aplicando prácticas ágiles basadas en **Scrum Framework (SCRUM)** [21] de forma adaptada al contexto de un proyecto académico individual. Este enfoque permitió una gestión flexible y adaptativa del proyecto. El trabajo se organizó en **Sprint** de dos semanas, donde se definieron objetivos claros y se realizaron revisiones periódicas para evaluar el progreso. Frente a enfoques estrictamente secuenciales, este enfoque resulta especialmente adecuado en proyectos de carácter experimental, como es el caso del modelado de temas, donde los resultados dependen en gran medida de la calidad del preprocesado y de la configuración de los modelos.

Para la planificación de tareas y realizar seguimiento del proyecto se utilizó la aplicación web Zube [47]. Esta herramienta se integra **Kanban**. Los gráficos burndown se utilizaron junto con el tutor en la reuniones de revisión de los sprints.

Para seguir el flujo de trabajo de **SCRUM** y facilitar el proceso de desarrollo, en este caso el tutor adoptó los roles de product owner y **SCRUM** master. Como evidencia de seguimiento, el tablero de Zube del proyecto está disponible en https://zube.io/tfg_topics/tfg-topics/. De forma complementaria, la trazabilidad de tareas e incidencias también puede consultarse en las *issues* del repositorio de GitHub del proyecto <https://github.com/zcc1001/tfg-topics/issues>.

4.2. Plataforma de Gestión de Ciclo de Vida del Software (DevOps)

Para la gestión del proyecto *TFG Topics* se adoptó la plataforma integral de GitHub. Esta infraestructura se configuró como núcleo operativo para soportar las prácticas del proceso de desarrollo adoptado. Las evidencias de ejecución pública se encuentran accesibles en el repositorio oficial del proyecto ejecución pueden consultarse en el repositorio oficial del proyecto <https://github.com/zcc1001/tfg-topics>.

A nivel operativo, el ecosistema de trabajo se articuló en las siguientes dimensiones:

- **Gestión de versiones de código:** Utilizando Git como sistema de control de versiones distribuido, se aplicó un modelo de ramificación que

favoreció la trazabilidad de cambios, el aislamiento por funcionalidades y la estabilidad de la rama principal [3].

- **Gestión de tareas y planificación ágil:** El seguimiento del *backlog* y de las iteraciones se instrumentó con GitHub Issues y su integración con los tableros Kanban de Zube, facilitando la vinculación entre tareas, incidencias y *commits*.
- **Integración continua y control de calidad:** Mediante GitHub Actions se automatizaron los flujos de trabajo *textitworkflows* para ejecutar baterías de pruebas unitarias (*testing*) y análisis estático de calidad de código en cada solicitud de extracción (*pull request*), asegurando la calidad del software.
- **Construcción de releases y despliegue:** Se definieron canalizaciones automatizadas para empaquetar versiones estables y sostener el despliegue continuo de los componentes de la aplicación.
- **Gestión del conocimiento y documentación:** Se utilizó GitHub Wiki para centralizar documentación técnica y funcional del sistema, estructurando de manera exhaustiva tanto el manual del programador —detallando la arquitectura hexagonal y la API— como el manual de usuario para la explotación de la interfaz en Streamlit.
- **Entornos de desarrollo virtualizados (GitHub Codespaces):** Se configuraron *devcontainers* para habilitar entornos estandarizados e independientes de la máquina local mediante GitHub Codespaces. Esta aproximación de **Entorno Como Código (EAC)** eliminó los conflictos de configuración de dependencias de Python y las librerías de modelado de temas. Asimismo, facilitó la comunicación síncrona y asíncrona con el tutor del proyecto, permitiendo desplegar de manera inmediata entornos efímeros pero completamente funcionales para testear y validar de forma conjunta los nuevos incrementos de software antes de su consolidación en producción.
- **Ingeniería de software asistida por IA:** GitHub Copilot se empleó como soporte de asistencia técnica en tareas de codificación, refactorización y revisión crítica de código, manteniendo supervisión humana en todo momento. Como evidencia, la incidencia pública #63 documenta su uso en revisiones y propuestas de mejora: <https://github.com/zcc1001/tfg-topics/issues/63>.

4.3. Herramientas de desarrollo

Entorno de desarrollo

El entorno de desarrollo utilizado fue **Visual Studio Code**, un editor de código fuente ligero y gratuito desarrollado por Microsoft [12]. Este entorno ofrece una amplia gama de extensiones y funcionalidades que facilitan la escritura, depuración y gestión de código en múltiples lenguajes de programación. Fue una herramienta esencial para el desarrollo de este proyecto gracias a su amplia integración con *plugins* que ayudan al desarrollo y depuración con el lenguaje de programación Python. Además, Visual Studio Code cuenta con integración nativa con GitHub, lo que facilita la gestión de versiones. Se valoró el uso de otros entornos como **PyCharm** [19], pero se optó por Visual Studio por su ligereza y flexibilidad.

También se utilizó **Jupyter Notebooks** [9] para la experimentación y análisis de datos, ya que permite combinar código, visualizaciones y documentación en un solo documento interactivo.

Lenguajes de programación

El lenguaje de programación principal utilizado en el desarrollo del proyecto fue **Python** [10]. Python es un lenguaje de alto nivel, interpretado y de propósito general, conocido por su sintaxis clara. Su elección se basa en la amplia gama de bibliotecas y **Framework** para el procesamiento de datos y modelado de temas disponibles.

Bibliotecas y frameworks

El núcleo del proyecto se apoya en técnicas de **NLP** aplicadas al análisis de documentos. Para el procesamiento de datos y modelado de temas se utilizaron diversas bibliotecas de Python, entre las que destacan:

- **Pandas** [16]: Biblioteca fundamental para la manipulación y análisis de datos.
- **NumPy** [7]: Biblioteca para computación científica y manejo de arreglos multidimensionales.
- **Gensim** [2]: Biblioteca especializada en modelado de temas y procesamiento de lenguaje natural.

- **NLTK** [6]: Biblioteca para el procesamiento de lenguaje natural, usado para tareas básicas tokenización y *stopwords*.
- **spaCy** [14]: Biblioteca avanzada para procesamiento más avanzado de lenguaje natural.

Sobre los textos preprocesados se aplican técnicas de modelado de temas. Se han estudiado y utilizado distintos enfoques, desde modelos probabilísticos clásicos hasta métodos más recientes basados en representaciones vectoriales. La comparación entre estas técnicas permite analizar sus fortalezas y limitaciones en función de criterios como la coherencia semántica de los temas, la interpretabilidad y el coste computacional.

Se utilizaron las siguientes bibliotecas de Python para el modelado de temas:

- **LDA** [5]: Implementación del modelo Latent Dirichlet Allocation para descubrir temas en grandes colecciones de documentos.
- **BERTopic** [13]: Biblioteca que utiliza modelos de lenguaje basados en transformadores para generar representaciones de documentos y realizar clustering.
- **Top2Vec** [23]: Biblioteca que combina incrustaciones de documentos y clustering para identificar temas en grandes conjuntos de datos.
- **FASTopic** [1]: Biblioteca que utiliza técnicas de reducción de dimensionalidad y clustering para el modelado eficiente de temas en grandes conjuntos de datos.

En cuanto a la experimentación con modelos y la optimización de hiperparámetros, se ha utilizado **Optuna** [8] como herramienta principal para la exploración del espacio de configuraciones. Esta biblioteca permite definir procesos de búsqueda automatizados y eficientes y facilita la comparación objetiva de los resultados mediante métricas cuantitativas como la coherencia de los temas. Frente a la selección manual de parámetros, este enfoque resulta más riguroso y contribuye a la reproducibilidad y trazabilidad de los experimentos realizados.

Finalmente, para la presentación y exploración de los resultados se ha desarrollado una interfaz web utilizando la librería **Streamlit** [15]. Su elección se debe principalmente a que esta herramienta permite construir interfaces web interactivas de forma ágil sin entrar en detalles complejos de

desarrollo web. Streamlit facilita la creación de paneles donde se pueden cargar datos y ajustar parámetros facilitando la visualización de los modelos generados y de sus métricas asociadas.

Otras herramientas

Además de las herramientas mencionadas anteriormente, se utilizaron diversas herramientas de apoyo orientadas a la gestión del entorno de ejecución y a la calidad del software desarrollado.

En primer lugar, se ha utilizado **Docker** [17] y **Docker Compose** como tecnología de **Contenedorización**. Docker permite encapsular la aplicación junto con todas sus dependencias en entornos reproducibles y aislados, lo que simplifica tanto el despliegue como la ejecución del sistema en diferentes máquinas. Este enfoque resulta especialmente útil ya que reduce problemas derivados de incompatibilidades de versiones y facilita la portabilidad y reproducibilidad de los experimentos.

Por otro lado, se han utilizado sistemas orientados a garantizar la calidad del código. En primer lugar, para el formateo automático del código se ha utilizado **Black**, lo que asegura un estilo de código uniforme en todo el proyecto. La organización y limpieza de importaciones se ha gestionado usando **isort**, mientras que el análisis estático y la detección de posibles errores o incumplimientos de estilo se han realizado con **flake8**. Estas herramientas mejoran la legibilidad del código y reducen la aparición de errores durante el desarrollo. A modo de síntesis, en la Tabla 4.1 se detalla el flujo integral de desarrollo, control de versiones y validación continua aplicado.

4.4. Documentación

Para la creación de la documentación del proyecto se utilizó **LaTeX** [4]. LaTeX es especialmente adecuado para la creación de documentos técnicos y científicos debido a su capacidad para manejar fórmulas matemáticas complejas, referencias bibliográficas y estructuras avanzadas de documentos. Además, LaTeX permite separar el contenido de la presentación, facilitando la gestión y el formato del documento final. Se optó por LaTeX frente a procesadores de texto tradicionales como Microsoft Word o Google Docs, ya que estos últimos pueden presentar limitaciones en la gestión de referencias y fórmulas complejas. Además, la universidad proporciona plantillas pre formateadas y estandarizadas para la creación de la memoria en LaTeX.

Etapas	Acciones principales	Herramientas
Planificación y seguimiento	Definición de tareas, priorización y revisión periódica del avance.	Zube, GitHub Issues.
Implementación	Desarrollo incremental por módulos y ejecución local o contenedorizada.	Python, Visual Studio Code, Docker, Makefile.
Control de versiones	Versionado del código, integración de cambios y trazabilidad de decisiones.	Git, GitHub.
Calidad estática	Formateo, orden de importaciones, análisis de estilo y verificación de tipos.	Black, isort, flake8, mypy, pre-commit, SonarQube.
Pruebas	Ejecución de pruebas unitarias por módulo y validación funcional de cambios.	Pytest.
Integración y despliegue	Ejecución reproducible del Pipeline completo en distintos modos.	Docker, Compose, GHCR , Makefile, GitHub actions.
Documentación	Redacción de memoria técnica y mantenimiento de documentación del proyecto.	LaTeX, Visual Studio Code, Wiki, markdown.

Tabla 4.1: Flujo de desarrollo, control de versiones y validación continua.

Como editor de LaTeX se utilizó Visual Studio Code con la extensión **LaTeX Workshop** [46].

5. Aspectos relevantes del desarrollo del proyecto

En este capítulo se describen los aspectos más relevantes y representativos del desarrollo del proyecto, poniendo el foco en las decisiones técnicas y de diseño adoptadas a lo largo de su ciclo de vida.

5.1. Arquitectura y diseño del software

Durante la fase de análisis y diseño del proyecto se tomaron decisiones clave. Debido a la naturaleza de los datos de entrada, documentos y textos que se obtienen de Github, provenientes de *issues*, *readmes*, y documentos en LaTeX de los distintos **TFG**, se decidió implementar un proceso específico de limpieza y normalización del texto, desacoplado del resto del sistema para facilitar su reutilización y evolución. También debido a la necesidad de experimentar con distintos enfoques de modelado de temas, se optó por diseñar una interfaz que permitiera la abstracción y así integrar diferentes proveedores de modelos. Estas necesidades influyeron directamente en el diseño del sistema, requiriendo una arquitectura flexible que permitiera integrar múltiples modelos sin modificar la lógica principal de la aplicación.

Durante la fase de análisis se identificaron claramente tres responsabilidades principales, que dieron lugar a la división del sistema en módulos independientes:

- Un módulo de **ingestión** (*backend-ingestion*), responsable de la obtención y persistencia de datos desde GitHub.

- Un módulo de **procesamiento** (*backend-processing*), encargado del preprocesado del texto, modelado de temas y evaluación.
- Un módulo de **visualización** (*frontend-web*), orientado a la exploración y análisis de los resultados generados.

Esta división facilitó el desarrollo incremental y permitió evolucionar cada módulo con un acoplamiento mínimo. En los módulos de backend se aplicó una arquitectura hexagonal [18], separando lógica de dominio, CUs y adaptadores de infraestructura. En la práctica, este enfoque mejoró la testabilidad, simplificó la sustitución de componentes y redujo el impacto de cambios en librerías externas.

5.2. Gestión de los datos y almacenamiento de resultados

Uno de los puntos críticos del desarrollo fue la elección de un formato común para el intercambio de información entre módulos. Para este propósito se optó por utilizar el formato **Parquet** [20], un formato de almacenamiento en columnas ampliamente utilizado en entornos de análisis de datos y *big data*.

La elección de Parquet frente a alternativas más simples como **Comma-Separated Values (CSV)** o formatos binarios genéricos se basó en varios criterios. En primer lugar, Parquet ofrece un almacenamiento más eficiente en espacio como en tiempo de acceso, gracias a su organización en columnas y a los mecanismos de compresión integrados. Este aspecto resulta especialmente relevante en un proyecto que genera múltiples ejecuciones de modelos, cada una con diferentes configuraciones de hiperparámetros y métricas asociadas.

La persistencia se organizó en dos niveles:

- **data/ingestion**: datasets base (*issues, readmes, thesis, abstracts, metadatos*).
- **data/processing/<modelo>**: artefactos de entrenamiento y evaluación por modelo/conjunto de datos (temas, métricas, parámetros, mejores hiperparámetros y ensayos de optimización).

Este contrato explícito de ficheros permitió desacoplar ejecución y análisis: el módulo web trabaja en modo lectura sobre resultados ya calculados, mientras que el procesamiento puede relanzarse sin afectar a la interfaz.

Desde el punto de vista de la implementación, el formato Parquet se utilizó principalmente para almacenar:

- Los datos sin procesar, incluyendo los textos y cualquier metadato relevante.
- Los resultados de la ejecución de los modelos de temas, incluyendo la información asociada a los temas generados.
- Las métricas de evaluación obtenidas en cada ejecución.
- Los valores de los hiperparámetros utilizados durante los procesos de optimización.

5.3. Optimización de hiperparámetros y gestión de experimentos

La fase de experimentación se apoyó en Optuna [8] para la búsqueda de hiperparámetros. Frente a herramientas más genéricas como *GridSearchCV* de *scikit-learn*, Optuna no impone restricciones sobre la estructura del modelo ni sobre el flujo de entrenamiento, lo que facilita su integración con bibliotecas heterogéneas como **LDA**, BERTopic, FASTopic o Top2Vec. Esta independencia del *framework* resulta clave en un proyecto que combina modelos con paradigmas y requisitos muy diferentes.

Los resultados de cada ejecución, junto con los parámetros utilizados y las métricas calculadas, se almacenan en formato Parquet. Esta decisión facilita la trazabilidad de los experimentos, permite analizar ejecuciones pasadas y evita la repetición innecesaria de cálculos costosos desde el punto de vista computacional.

5.4. Despliegue y modos de ejecución

Durante la implementación se priorizó que el proyecto pudiera ejecutarse tanto en contextos de uso final como en entornos de desarrollo. Para ello se definió una orquestación centralizada con **Makefile** y Docker Compose [17].

Un aspecto destacable de este **TFG** ha sido la aplicación de prácticas de gestión del ciclo de desarrollo ya descritas en el capítulo de Técnicas y herramientas: control de versiones con Git, planificación y seguimiento con GitHub Issues y Zube, automatización de validaciones mediante integración continua,

y gestión de versiones de entrega. Estas prácticas se integraron directamente con la estrategia de despliegue para garantizar trazabilidad y reproducibilidad del sistema. Como evidencia pública del proceso puede consultarse el repositorio oficial del proyecto en <https://github.com/zcc1001/tfg-topics>. Para más detalle operativo sobre herramientas y flujos, véase el Capítulo 4.

En este contexto, **GitHub Actions** se utilizó para automatizar *workflows* de validación continua sobre cambios de código e integraciones, ejecutando pruebas y verificaciones de calidad. Asimismo, **GHCR** se empleó como registro de contenedores para publicar imágenes versionadas consumidas por el modo **MODE=release**, reduciendo el tiempo de puesta en marcha en entornos nuevos. Por último, **GitHub Releases** se utilizó para gestionar versiones estables del proyecto y centralizar artefactos y notas de versión, reforzando la trazabilidad de cada entrega.

El flujo de trabajo contempla tres modos, cuya arquitectura de decisión se ilustra en la Figura 5.1:

- **MODE=release**: uso de imágenes precompiladas en **GHCR**, orientado a ejecución rápida.
- **MODE=docker**: construcción local de imágenes para validar cambios de infraestructura.
- **MODE=local**: ejecución nativa en Python para depuración y desarrollo iterativo.

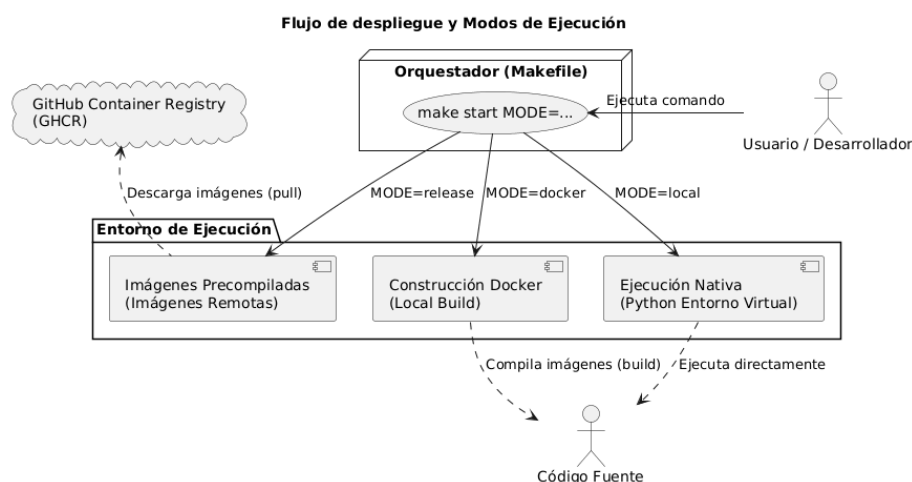


Figura 5.1: Diagrama conceptual de los modos de ejecución del proyecto.

Esta estrategia redujo fricción en la puesta en marcha y mejoró la reproducibilidad de resultados entre equipos y sistemas operativos.

5.5. Interfaz web y exploración de resultados

El módulo de visualización se implementó como un frontal web utilizando Streamlit [15]. Este componente consume directamente los resultados generados por el módulo de procesamiento y almacenados en ficheros Parquet, manteniendo así un acoplamiento mínimo entre ambos módulos.

La interfaz permite seleccionar modelos, explorar temas generados, visualizar métricas y analizar resultados de forma interactiva, tal y como se puede apreciar en la Figura 5.2. Esta decisión sustituye salidas estáticas por una herramienta más adecuada para un análisis exploratorio y facilita tanto la evaluación del sistema como la comprensión de los resultados obtenidos.

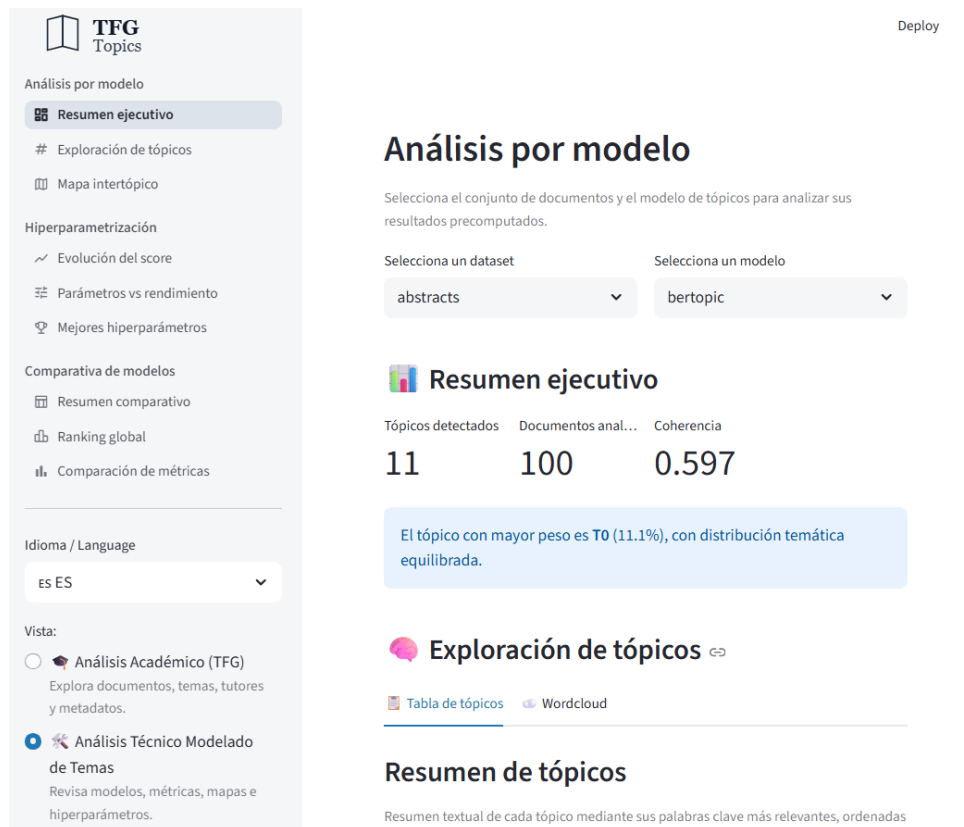


Figura 5.2: Vista general de la interfaz interactiva desarrollada con Streamlit.

6. Trabajos relacionados

No se han encontrado trabajos relacionados que integren de forma conjunta las funcionalidades propuestas en este TFG, combinando técnicas de modelado de temas con procesos de ingesta y análisis de información textual obtenida desde repositorios de GitHub.

Sin embargo, sí existen diversos Trabajo de Fin de Grado relacionados parcialmente con algunas de las áreas abordadas en este proyecto, especialmente en el ámbito de la extracción de información desde GitHub, la minería de texto y el procesamiento del lenguaje natural.

En relación con la extracción y análisis de información procedente de repositorios GitHub, se han identificado varios trabajos desarrollados en cursos anteriores.

El primero de ellos, presentado en junio de 2019, lleva por título *Auto-evaluación del proceso de desarrollo de Trabajo Fin de Grado en GitHub*, desarrollado por Roberto Luquero y disponible en el repositorio <https://github.com/rlp0019/Activiti-Api>. Este proyecto extrae métricas cuantitativas obtenidas tanto del sistema de control de versiones Git como del sistema de gestión de incidencias (*issues*), utilizando posteriormente dichos valores para compararlos con umbrales predefinidos con fines de evaluación académica.

Posteriormente, en junio de 2022, Joaquín García Molina presentó el trabajo *Comparador de métricas de evolución en repositorios software*, disponible en https://github.com/Joaquin-GM/GII_0_MA_19.07-Comparador-de-metricas-de-evolucion-en-repositorios-Software. Este proyecto sigue una aproximación similar, centrada en la obtención y comparación de métricas cuantitativas asociadas a la evolución de repositorios software mediante información procedente de Git y del sistema de incidencias.

En junio de 2023 se desarrolló el TFG *Evaluación de simulaciones de proyectos en GitHub orientadas a docencia*, realizado por Licinio Fernández Maurelo y disponible en <https://github.com/lfx1001/>. A diferencia de los trabajos anteriores, este proyecto se centra en el análisis completo del ciclo de vida asociado a la resolución de incidencias, comparando distintos flujos de trabajo para determinar el grado de similitud entre ellos.

Los tres trabajos anteriores comparten un enfoque orientado al ámbito docente, proporcionando herramientas de apoyo para que los estudiantes puedan evaluar y comparar sus procesos de desarrollo realizados en GitHub frente a proyectos considerados de referencia o de alta calidad.

En el contexto de la minería de texto, en enero de 2022 se presentó el proyecto *Plataforma de experimentación de text mining sobre repositorios de código abierto GitHub*, desarrollado por Pablo Fernández Bravo y disponible en <https://github.com/2021-22-TFG-ene/github-text-mining-tfg>. Este trabajo emplea la información contenida en las incidencias de los repositorios para realizar tareas de clasificación automática de texto, análisis de sentimientos y generación de resúmenes.

Finalmente, dentro del ámbito del resumen automático de textos mediante técnicas de procesamiento del lenguaje natural, en febrero de 2021 se presentó el TFG *JIZT. Servicio de Resumen de Textos con AI en la Nube*, desarrollado por Diego de Miguel Lozano y disponible en <https://github.com/dmlls/jizt-tfg>. Este proyecto se centra en la generación automática de resúmenes textuales mediante técnicas de inteligencia artificial aplicadas al procesamiento del lenguaje natural.

Tabla 6.1: Resumen de trabajos relacionados

Año	Trabajo	Área principal	Relación con el TFG
2019	<i>Autoevaluación del proceso de desarrollo de Trabajo Fin de Grado en GitHub</i>	Métricas sobre Git e incidencias	Extracción y análisis cuantitativo de repositorios GitHub
2021	<i>JIZT. Servicio de Resumen de Textos con AI en la Nube</i>	Resumen automático de texto	Procesamiento del lenguaje natural
2022	<i>Comparador de métricas de evolución en repositorios software</i>	Métricas de evolución software	Comparación de métricas de repositorios
2022	<i>Plataforma de experimentación de text mining sobre repositorios de código abierto GitHub</i>	Minería de texto	Clasificación, análisis de sentimientos y resumen
2023	<i>Evaluación de simulaciones de proyectos en GitHub orientadas a docencia</i>	Simulación y comparación de flujos de trabajo	Análisis del ciclo de vida de incidencias

7. Conclusiones y Líneas de trabajo futuras

El proyecto *TFG Topics* ha permitido diseñar e implementar una solución completa para el análisis temático cualitativo de memorias de **TFG** del grado en Ingeniería Informática de la Universidad de Burgos publicadas en GitHub, cubriendo todo el ciclo de vida del dato: ingesta, procesamiento, modelado y visualización.

A nivel de objetivos, se considera que el proyecto se ha cumplido de forma satisfactoria. Se ha construido un **Pipeline** modular con tres bloques bien diferenciados (*backend-ingestion*, *backend-processing* y *frontend-web*), con una arquitectura hexagonal en backend que facilita la mantenibilidad, la escalabilidad y la sustitución de componentes. Además, se ha definido un contrato de datos estable en formato Parquet, lo que mejora la trazabilidad experimental y desacopla claramente la generación de resultados de su consumo en la interfaz.

Desde el punto de vista experimental, se han entrenado y comparado cuatro enfoques de modelado de temas (**LDA**, BERTopic, Top2Vec y FAS-Topic) sobre cuatro fuentes textuales (*abstracts*, *issues*, *readmes* y cuerpo de las memorias). En el estado actual del proyecto, BERTopic ofrece la mejor coherencia en *issues*, *readmes* y cuerpo de las memorias, mientras que Top2Vec destaca en *abstracts*. Por su parte, LDA presenta un comportamiento competitivo en varios escenarios con un coste computacional significativamente menor. Este resultado evidencia un compromiso técnico relevante entre calidad semántica e inversión de recursos.

Estos resultados confirman que no existe un único modelo óptimo universal, sino que el rendimiento depende del tipo de texto analizado y de

su granularidad semántica. También se observa un compromiso claro entre calidad temática y coste computacional: los modelos basados en *embeddings* suelen mejorar la coherencia en varios escenarios, a costa de mayores tiempos de ejecución.

Como valoración crítica, identificamos tres limitaciones principales. En primer lugar, la evaluación se apoya mayoritariamente en métricas automáticas de coherencia, que no sustituyen una validación cualitativa experta. En segundo lugar, la heterogeneidad de los repositorios de origen introduce ruido documental y estructural que afecta a la estabilidad del modelado. En tercer lugar, algunos modelos basados en *embeddings* presentan costes de ejecución elevados, lo que condiciona la escalabilidad en corpus más grandes.

Además de los resultados cuantitativos, uno de los aportes principales del trabajo es haber construido una base software reutilizable: el sistema permite reejecutar fases de forma independiente, comparar configuraciones de manera homogénea y exponer resultados en una interfaz interactiva orientada a análisis exploratorio.

7.1. Líneas de trabajo futuras

Como evolución natural del proyecto, se plantean las siguientes líneas:

- Mejora de la interpretación de los temas generados. Actualmente, los temas obtenidos por los modelos se representan mediante distribuciones de probabilidad sobre conjuntos de términos, lo que requiere una interpretación posterior por parte del usuario para identificar de manera precisa la temática asociada. Habitualmente, esta tarea se realiza mediante validación humana. Como línea de trabajo futura, se plantea incorporar modelos grandes de lenguaje (**Large Language Model (LLM)**) para automatizar el proceso de etiquetado e interpretación semántica de los temas, ya que las pruebas preliminares realizadas han mostrado resultados satisfactorios en tareas de clasificación e identificación temática.
- Ampliar la evaluación con más métricas automáticas (por ejemplo, diversidad temática y estabilidad entre ejecuciones) y reforzar la validación humana con protocolos estandarizados de evaluación.
- Incorporar análisis temporal para estudiar la evolución de temáticas por curso académico y detectar tendencias emergentes.

- Mejorar la normalización lingüística en español (tratamiento de tecnicismos, acrónimos y expresiones de dominio) para aumentar la calidad semántica de los temas.
- Explorar variantes de modelado adicionales (modelos jerárquicos, dinámicos o nuevas configuraciones de *sentence-transformers*).
- Optimizar rendimiento mediante caché de *embeddings*, ejecución paralela y otras estrategias de reducción de coste computacional.
- Mejorar la interfaz de usuario: filtros académicos más ricos, exportación de informes para toma de decisiones y mejoras en accesibilidad para usuarios con necesidades especiales.

En definitiva, este TFG no solo entrega un prototipo funcional, sino una base técnica sólida y extensible sobre la que continuar investigación aplicada en analítica temática de documentación académica.

Acrónimos

AI Artificial Intelligence. 9, 11

ANN Artificial Neural Networks. 15

BERT Bidirectional Encoder Representations from Transformers. 16, 18, 22

BOW Bag of Words. 2, 14, 15, 16

c-TF-IDF Class-based Term Frequency-Inverse Document Frequency. 18, 21

CSV Comma-Separated Values. 34

CU Caso de Uso. 34

DSR Reconstrucción de Relaciones Semánticas Dual. 18

EAC Entorno Como Código. 27

ECTS European Credit Transfer and Accumulation System. 12

EPS Escuela Politécnica Superior. 12

ETP Embedding Transport Plan. 19

GHCR GitHub Container Registry. 31, 36

HDBSCAN Hierarchical Density-Based Spatial Clustering of Applications with Noise. 16, 18, 19, 21, 22

- LDA** Latent Dirichlet Allocation. [I](#), [II](#), [2](#), [6](#), [16](#), [17](#), [20](#), [21](#), [29](#), [35](#), [43](#)
- LLM** Large Language Model. [44](#)
- NLP** Natural Language Processing. [I](#), [II](#), [9](#), [10](#), [11](#), [13](#), [14](#), [28](#)
- NLTK** Natural Language Tool Kit. [14](#)
- SCRUM** Scrum Framework. [26](#)
- TF** Term Frequency. [14](#)
- TF-IDF** Term Frequency-Inverse Document Frequency. [14](#)
- TFG** Trabajo de Fin de Grado. [I](#), [II](#), [1](#), [2](#), [5](#), [6](#), [9](#), [10](#), [12](#), [16](#), [22](#), [33](#), [35](#), [39](#), [43](#), [45](#)
- UMAP** Uniform Manifold Approximation and Projection. [16](#), [18](#), [21](#), [22](#)

Glosario

Clustering Proceso de agrupación automática de elementos en conjuntos homogéneos basados en medidas de similitud. [16](#), [17](#), [18](#), [19](#), [21](#), [22](#)

Coherencia Métrica que evalúa la relación semántica entre los términos más representativos de un tema. [17](#), [20](#)

Contenedorización Técnica de virtualización ligera que encapsula una aplicación y sus dependencias en entornos aislados. [30](#)

Corpus Conjunto estructurado de documentos utilizado como base para el análisis lingüístico o estadístico. [14](#), [20](#)

Distancia euclidiana Métrica que calcula la distancia geométrica directa entre dos puntos en un espacio vectorial. [15](#)

Doc2Vec Modelo de aprendizaje no supervisado que genera representaciones vectoriales de documentos completos. [22](#)

Embeddings Representación vectorial (numérica) del significado semántico de las palabras en un espacio multidimensional. [15](#), [16](#), [17](#), [18](#), [19](#), [20](#), [21](#), [22](#)

FastText Modelo de representaciones vectoriales que incorpora información de subpalabras para mejorar el tratamiento de palabras raras. [22](#)

Framework Estructura de software que proporciona una base sobre la que desarrollar aplicaciones de forma estandarizada. [28](#)

Grid Search Método de búsqueda de hiperparámetros que explora una parrilla de valores predefinida. [23](#)

- Hiperparametrización** Proceso de configuración y ajuste de los hiperparámetros de un modelo para optimizar su rendimiento. 22
- K-Means** Algoritmo de clustering que agrupa datos en k conjuntos minimizando la distancia interna a los centroides. 22
- Kanban** Sistema visual de gestión del trabajo que permite controlar el flujo de tareas mediante columnas y estados. 26
- Lematización** Proceso de reducir las palabras a su forma canónica o lema. 9, 12, 13, 17
- Modelado de temas** Técnica de aprendizaje no supervisado para encontrar temas subyacentes en un corpus de documentos. 2, 15
- Optimización Bayesiana** Método de búsqueda de hiperparámetros que utiliza un modelo probabilístico para encontrar los valores óptimos de forma eficiente. 23
- Pipeline** Cadena de procesamiento secuencial de datos estructurada en etapas, donde la salida de cada fase actúa como entrada de la siguiente. v, 11, 31, 43
- Preprocesamiento de texto** Conjunto de técnicas aplicadas a textos sin estructurar para limpiarlos, normalizarlos y prepararlos para su análisis automático. 13
- Random Search** Método de búsqueda de hiperparámetros que explora combinaciones aleatorias en un espacio de búsqueda. 23
- Reducción de dimensionalidad** Técnica que reduce el número de variables de un conjunto de datos preservando la información relevante. 16, 17, 18, 19, 21, 22
- Similitud del coseno** Métrica que mide la similitud entre dos vectores calculando el coseno del ángulo que los separa. 15
- Sprint** Iteración temporal fija dentro de una metodología ágil en la que se desarrollan y revisan funcionalidades concretas. 26
- Stopwords** Palabras vacías con bajo valor semántico que se eliminan en la fase de preprocesamiento. 9, 13, 14, 17

Tokenización Proceso de dividir texto en unidades mínimas (tokens). [9](#), [11](#), [12](#), [13](#)

Ventana deslizante Técnica que analiza secuencias de elementos consecutivos dentro de un texto para estudiar coocurrencias. [20](#)

Bibliografía

- [1] Fastopic: FASTopic. <https://github.com/bobxwu/fastopic>.
- [2] Gensim: Topic modelling for humans. https://radimrehurek.com/gensim/auto_examples/index.html#documentation.
- [3] Git - Reference. <https://git-scm.com/docs>.
- [4] LaTeX - A document preparation system. <https://www.latex-project.org/>.
- [5] Lda: Topic modeling with latent Dirichlet Allocation — lda documentation. <https://lda.readthedocs.io/en/latest/>.
- [6] NLTK :: Natural Language Toolkit. <https://www.nltk.org/>.
- [7] NumPy user guide — NumPy v1.26 Manual. <https://numpy.org/doc/1.26/user/index.html#user>.
- [8] Optuna - A hyperparameter optimization framework. <https://optuna.org/>.
- [9] Project Jupyter Documentation — Jupyter Documentation 4.1.1 alpha documentation. <https://docs.jupyter.org/en/latest/>.
- [10] Python 3.10.19 Documentation. <https://docs.python.org/3.10/>.
- [11] ¿Qué es la tokenización? Tipos, casos de uso, aplicación. <https://www.datacamp.com/blog/what-is-tokenization>.
- [12] ¿Qué es Visual Studio Code y cuáles son sus ventajas? | Blog de Arsys | Blog de Arsys. <https://www.arsys.es/blog/que-es-visual-studio-code-y-cuales-son-sus-ventajas>.

- [13] Quick Start - BERTopic. https://maartengr.github.io/BERTopic/getting_started/quickstart/quickstart.html.
- [14] spaCy 101: Everything you need to know · spaCy Usage Documentation. <https://spacy.io/usage/spacy-101>.
- [15] Streamlit Docs. <https://docs.streamlit.io/>.
- [16] User Guide — pandas 2.3.3 documentation. https://pandas.pydata.org/docs/user_guide/index.html.
- [17] Docker manuals. <https://docs.docker.com/manuals/>, 09:11:23 +0100 +0100.
- [18] Hexagonal Architecture and Clean Architecture (with examples). <https://dev.to/dyarleniber/hexagonal-architecture-and-clean-architecture-with-examples-48oi>, 2022.
- [19] PyCharm : Todo sobre el IDE de Python más popular. <https://datascientest.com/es/pycharm>, 2022.
- [20] Apache Parquet documentation. <https://parquet.apache.org/docs/>, 2026.
- [21] Agile Alliance. Agile 101 - What is Agile? <https://www.agilealliance.org/agile101/>, 2024. [En línea; consultado el 19-septiembre-2024].
- [22] Dimo Angelov. Top2Vec: Distributed Representations of Topics, 2020. <http://arxiv.org/abs/2008.09470>.
- [23] Dimo Angelov. Ddangelov/Top2Vec. <https://github.com/ddangelov/Top2Vec>, 2025.
- [24] John Atkinson-Abutridy. *Analítica textual*. Marcombo.
- [25] John Atkinson-Abutridy. *Grandes modelos de lenguaje. Conceptos, técnicas y aplicaciones*. Marcombo, 2023.
- [26] Aishwarya Bhangale. Introduction to Topic Modelling with LDA, NMF, Top2Vec and BERTopic, 2023. <https://medium.com/blend360/introduction-to-topic-modelling-with-lda-nmf-top2vec-and-bertopic-ffc3624d44e4>, [Internet, descargado 10-noviembre-2025].

- [27] Iqra Bismi. Topic modelling using LDA, 2023. <https://medium.com/@iqra.bismi/topic-modelling-using-lda-fe81a2a806e0>, [Internet, descargado 10-noviembre-2025].
- [28] David M Blei, Andrew Y Ng, and Michael I Jordan. Latent dirichlet allocation. *Journal of Machine Learning Research*, 3:993–1022, 2003.
- [29] Castillo-Velásquez Francisco Antonio Campos-Sánchez Jonathan Josefatz, Gonzalez-Gutierrez Fidel. Revisión Sistemática de Técnicas de Tokenización para la Clasificación de Información en Inteligencia Artificial. https://virtual.cuautitlan.unam.mx/intar/memoriasceiaait/wp-content/uploads/sites/19/2024/12/24-Revision-Sistemática-de-Técnicas-de-Tokenización-__para-la-Clasificación-de-Información-en-Inteligencia-Artificial-EDITADO.pdf.
- [30] Gustavo Espíndola. ¿Qué son los embeddings y cómo se utilizan en la inteligencia artificial con python?, 2023. <https://gustavo-espindola.medium.com/qu%C3%A9-son-los-embeddings-y-c%C3%B3mo-se-utilizan-en-la-inteligencia-artificial-con-python-45b751ed86a5>.
- [31] Luca Franceschi, Michele Donini, Valerio Perrone, Aaron Klein, Cédric Archambeau, Matthias Seeger, Massimiliano Pontil, and Paolo Frasconi. Hyperparameter optimization in machine learning. *Foundations and Trends® in Machine Learning*, 18(6):1054–1201, 2025. <http://dx.doi.org/10.1561/22000000088>.
- [32] Thiyaneshwaran G. Top2vec for Topic Modeling and Semantic Similarity and Search, 2022. <https://medium.com/@ThiyaneshwaranG/top2vec-for-topic-modeling-and-semantic-similarity-and-search-77db82dda7d3>.
- [33] Lin Gan, Tao Yang, Yifan Huang, Boxiong Yang, Yami Yanwen Luo, Lui Wing Cheung Richard, and Dabo Guo. Experimental comparison of three topic modeling methods with lda, top2vec and bertopic. In *International Symposium on Artificial Intelligence and Robotics*, pages 376–391. Springer, 2023.
- [34] gewarren. Descripción de los tokens - .NET. <https://learn.microsoft.com/es-es/dotnet/ai/conceptual/understanding-tokens>.

- [35] Thomas L Griffiths and Mark Steyvers. Finding scientific topics. *PNAS*, 101(suppl 1):5228–5235, 2004.
- [36] Maarten Grootendorst. BERTopic: Neural topic modeling with a class-based TF-IDF procedure, 2022. <http://arxiv.org/abs/2203.05794>.
- [37] Mustapha Hankar, Mohammed Kasri, and Abderrahim Beni-Hssane. A comprehensive overview of topic modeling: Techniques, applications and challenges. *Neurocomputing*, 628:129638, 2025. <https://linkinghub.elsevier.com/retrieve/pii/S0925231225003108>.
- [38] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd edition, 2025. <https://web.stanford.edu/~jurafsky/slp3/>.
- [39] Carlos López Nozal, Raúl Marticorena Sánchez, Juan José Rodríguez, and Andrés Bustillo Iglesias. Proceso de gestión de trabajos fin de carrera. In *Jornadas de Enseñanza Universitaria de la Informática*, pages 413–420. Jornadas de Enseñanza Universitaria de la Informática, 2009.
- [40] Madhurima Nath. Topic modeling algorithms, 2023. <https://medium.com/@m.nath/topic-modeling-algorithms-b7f97cec6005>, [Internet; descargado 10-noviembre-2025].
- [41] João Pedro. Understanding Topic Coherence Measures. <https://towardsdatascience.com/understanding-topic-coherence-measures-4aa41339634c/>, 2022.
- [42] Serhad Sarica and Jianxi Luo. Stopwords in technical language processing. *Plos one*, 16(8), 2021. <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0254937>.
- [43] Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. Practical bayesian optimization of machine learning algorithms, 2012. <https://arxiv.org/abs/1206.2944>.
- [44] Dr Tiya Vaj. How to evaluate novel topic modeling method. <https://vtiya.medium.com/how-to-evaluate-novel-topic-modeling-method-104ad9684428>, 2023.
- [45] Xiaobao Wu, Thong Nguyen, Delvin Ce Zhang, William Yang Wang, and Anh Tuan Luu. FASTopic: Pretrained Transformer is a Fast, Adaptive, Stable, and Transferable Topic Model. *arXiv e-prints*, 2024.

- [46] James Jianqiao Yu. James-Yu/LaTeX-Workshop. <https://github.com/James-Yu/LaTeX-Workshop>, 2025.
- [47] Zube. Zube - project management for agile development. <https://zube.io>. Accessed: 2024-09-19.